Explicación línea por línea — Guía de Laboratorio N.° 3

Tecnologías Emergentes (ISO46B) — Automatización y Calidad de Código

Este documento explica, línea por línea, el código de `organizador.py` (versión final documentada), `test_organizador.py`, y los comandos de terminal usados en la guía. La idea es que puedas dictar la sesión sin depender de memoria: cada fragmento tiene el "qué hace" y el "por qué está ahí".

---

1. `organizador.py` — el script de automatización

1.1 Encabezado e imports

```python
"""Organiza los archivos de una carpeta en subcarpetas segun su extension."""
import argparse
import shutil
from pathlib import Path
```

- **Línea 1** — Es el *docstring de módulo*: el primer string literal del archivo. Python lo guarda en `organizador.__doc__`. Herramientas como `pydoc` o `help(organizador)` lo muestran como descripción del archivo completo. Por PEP 257, va entre comillas triples dobles.
- **`import argparse`** — Trae la librería estándar para construir interfaces de línea de comandos (leer argumentos como `carpeta_prueba --dry-run`).
- **`import shutil`** — Trae las operaciones de archivo de "alto nivel" (aquí, específicamente `shutil.move`).
- **`from pathlib import Path`** — Trae la clase `Path`, que representa una ruta del sistema de archivos como objeto (no como texto). Se importa así, y no `import pathlib`, para poder escribir `Path` en vez de `pathlib.Path` en el resto del archivo.

1.2 El diccionario de categorías

```python
CATEGORIAS = {
    ".pdf": "Documentos", ".docx": "Documentos", ".txt": "Documentos",
    ".jpg": "Imagenes", ".jpeg": "Imagenes", ".png": "Imagenes",
    ".mp4": "Videos", ".mov": "Videos",
    ".zip": "Comprimidos", ".rar": "Comprimidos",
}
```

- Es una constante a nivel de módulo (por convención, en `MAYÚSCULAS`). Es un diccionario Python normal: la **clave** es la extensión del archivo (en minúsculas, con el punto), el **valor** es el nombre de la subcarpeta destino.
- No es una función: se calcula una sola vez al importar el archivo, no cada vez que se organiza una carpeta. Por eso vive fuera de cualquier función.
- Está incompleto a propósito — el estudiante puede (y debería) agregar más extensiones si lo necesita.

1.3 La función `clasificar`

python

```python
def clasificar(archivo: Path) -> str:
    """Determina la categoria de un archivo segun su extension.

    Args:
        archivo: Ruta del archivo a clasificar.

    Returns:
        Nombre de la categoria correspondiente, u "Otros" si la
        extension no esta registrada.
    """
    return CATEGORIAS.get(archivo.suffix.lower(), "Otros")
```

- `archivo: Path` — El *type hint* dice que el parámetro `archivo` debe ser un objeto `Path` (no una cadena de texto). No es obligatorio en tiempo de ejecución (Python no lo verifica solo), pero el editor y herramientas como `mypy` sí lo usan para detectar errores.
- `-> str` — Otro type hint: indica que la función siempre devuelve una cadena de texto.
- El docstring, en estilo Google: una línea de resumen, línea en blanco, sección `Args:` con el parámetro, línea en blanco, sección `Returns:` con lo que devuelve. Nota que **no** repite el tipo (`Path`, `str`) dentro del docstring — eso ya lo dicen los type hints de la firma, por eso no se duplica (esto es exactamente el punto de la diapositiva 12 del guion).
- `archivo.suffix` — Atributo de `Path` que devuelve la extensión del archivo incluyendo el punto (por ejemplo, `.JPG`). Si el archivo no tiene extensión, devuelve una cadena vacía.
- `.lower()` — Convierte la extensión a minúsculas, para que `.JPG` y `.jpg` se traten igual.
- `CATEGORIAS.get(clave, "Otros")` — El método `.get()` de un diccionario busca la clave; si no la encuentra, en vez de lanzar un `KeyError` devuelve el segundo

argumento (`"Otros"`). Por eso cualquier extensión no registrada cae en "Otros" en lugar de romper el programa.

1.4 La función `organizar_carpeta`

python

```python
def organizar_carpeta(carpeta: Path, simulacion: bool = False) -> int:
    """Organiza los archivos de una carpeta en subcarpetas por categoria.

    Args:
        carpeta: Carpeta cuyos archivos se van a organizar.
        simulacion: Si es True, solo muestra que haria sin mover archivos.

    Returns:
        Numero de archivos movidos (o que se moverian, en modo simulacion).
    """
    movidos = 0
    for archivo in carpeta.iterdir():
        if archivo.is_dir() or archivo.name.startswith("."):
            continue
        categoria = clasificar(archivo)
        destino_carpeta = carpeta / categoria
        destino = destino_carpeta / archivo.name

        if simulacion:
            print(f"[SIMULACION] {archivo.name} -> {categoria}")
        else:
            destino_carpeta.mkdir(exist_ok=True)
            shutil.move(str(archivo), str(destino))
            print(f"Movido: {archivo.name} -> {categoria}")
        movidos += 1
    return movidos
```

Línea por línea:

- `simulacion: bool = False` — Segundo parámetro, con **valor por defecto**. Esto significa que se puede llamar `organizar_carpeta(carpeta)` sin el segundo argumento, y Python asume `False`. Es lo que permite que el modo normal y el modo `--dry-run` compartan la misma función.
- `movidos = 0` — Contador que se irá incrementando; es lo que la función finalmente devuelve.
- `for archivo in carpeta.iterdir():` — `.iterdir()` es un método de `Path` que devuelve un objeto iterable con cada elemento (archivo o carpeta) dentro de `carpeta`, uno por uno. El bucle procesa cada elemento como `archivo`.

- `if archivo.is_dir() or archivo.name.startswith("."):` — Dos condiciones combinadas con `or`. `.is_dir()` es `True` si el elemento es una carpeta (no queremos "organizar" las subcarpetas que el propio script ya creó, como `Imagenes`). `.name.startswith(".")` es `True` para archivos ocultos (por ejemplo, `.gitkeep` o `.DS_Store` en macOS). Si cualquiera de las dos es verdadera, `continue` salta a la siguiente iteración sin hacer nada más con ese elemento.
- `categoria = clasificar(archivo)` — Reutiliza la función que ya explicamos.
- `destino_carpeta = carpeta / categoria` — El operador `/` está *sobrecargado* en `Path`: no es división, es concatenación de rutas. Si `carpeta` es `Path("carpeta_prueba")` y `categoria` es `"Imagenes"`, el resultado es `Path("carpeta_prueba/Imagenes")` (o con barras invertidas en Windows, automáticamente).
- `destino = destino_carpeta / archivo.name` — Misma lógica: la ruta completa de destino, incluyendo el nombre del archivo.
- `if simulacion:` — Si el modo simulación está activo, **solo imprime** lo que haría, sin tocar el disco. Esto es lo que hace que `--dry-run` sea seguro de ejecutar cualquier cantidad de veces.
- `else:` — Rama real:
  - `destino_carpeta.mkdir(exist_ok=True)` — Crea la subcarpeta destino si no existe. `exist_ok=True` evita que Python lance un error si la carpeta ya fue creada en una ejecución anterior (sin ese argumento, `mkdir()` falla si la carpeta ya existe).
  - `shutil.move(str(archivo), str(destino))` — Mueve el archivo. Nótese `str(archivo)` y `str(destino)`: `shutil.move` en versiones antiguas de Python esperaba cadenas de texto, no objetos `Path`; convertirlos explícitamente evita problemas de compatibilidad. (En Python moderno acepta ambos, pero convertir es más seguro y explícito).
  - El `print` de confirmación, para que quien ejecute el script vea qué pasó.
- `movidos += 1` — Se incrementa el contador tanto en modo simulación como en modo real, porque en ambos casos el archivo "fue procesado" (movido o descrito).
- `return movidos` — Al terminar el bucle, la función devuelve cuántos archivos procesó. Esto es lo que `main()` usa para imprimir el total.

1.5 La función `main` (interfaz de línea de comandos)

```
python
```

```python
def main() -> None:
    """Punto de entrada de linea de comandos del organizador."""
    parser = argparse.ArgumentParser(description="Organiza archivos por tipo.")
    parser.add_argument("carpeta", type=Path, help="Carpeta a organizar")
    parser.add_argument(
        "--dry-run", action="store_true", help="Solo simula, no mueve archivos"
    )
    args = parser.parse_args()

    total = organizar_carpeta(args.carpeta, simulacion=args.dry_run)
    print(f"Total de archivos procesados: {total}")


if __name__ == "__main__":
    main()
```

- `-> None` — Type hint que indica que la función no devuelve ningún valor útil (solo imprime y termina).
- `argparse.ArgumentParser(description=...)` — Crea el objeto que va a leer los argumentos de la terminal. El `description` aparece si alguien ejecuta `python organizador.py --help`.
- `parser.add_argument("carpeta", type=Path, ...)` — Define un argumento **posicional** (sin guiones al inicio, por eso es obligatorio y va en un orden fijo). `type=Path` le dice a `argparse` que convierta automáticamente el texto recibido (por ejemplo, `"carpeta_prueba"`) en un objeto `Path`, para no tener que convertirlo manualmente después.
- `parser.add_argument("--dry-run", action="store_true", ...)` — Define un argumento **opcional** (empieza con `--`). `action="store_true"` significa: si la bandera está presente, el valor es `True`; si no está, es `False`. No recibe un valor después (no es `--dry-run algo`, es solo `--dry-run`).
- `args = parser.parse_args()` — Lee los argumentos reales que se pasaron en la terminal y los guarda como atributos del objeto `args`.
- `args.carpeta` — Atributo correspondiente al argumento posicional `"carpeta"`.
- `args.dry_run` — **Atención aquí**: el argumento se definió como `"--dry-run"` (con guion), pero `argparse` reemplaza automáticamente los guiones por guiones bajos al crear el atributo. Por eso es `args.dry_run`, no `args.dry-run` (que ni siquiera sería válido en Python, porque el guion se interpretaría como una resta).
- `organizar_carpeta(args.carpeta, simulacion=args.dry_run)` — Llama a la función que ya explicamos, pasando la carpeta ingresada por el usuario y traduciendo la bandera `dry_run` al parámetro `simulacion`.

- `if __name__ == "__main__":` — Este es un patrón estándar de Python. `__name__` vale `"__main__"` solo cuando el archivo se ejecuta directamente (`python organizador.py ...`); si en cambio el archivo se **importa** desde otro (como hace `test_organizador.py` con `from organizador import clasificar, organizar_carpeta`), `__name__` vale `"organizador"`, y por lo tanto `main()` **no se ejecuta automáticamente**. Esto es exactamente lo que permite que las pruebas importen `clasificar` y `organizar_carpeta` sin que se dispare la interfaz de línea de comandos.

---

2. `test_organizador.py` — las pruebas

```python
from organizador import clasificar, organizar_carpeta
```

- Importa únicamente las dos funciones que se van a probar, directamente desde el módulo `organizador` (el archivo `organizador.py` en la misma carpeta). Por eso `pytest` debe ejecutarse desde la raíz del repositorio: si se ejecuta desde otra carpeta, Python no encuentra el módulo `organizador`.

2.1 Primera prueba

```python
def test_clasificar_reconoce_imagen(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")
    assert clasificar(archivo) == "Imagenes"
```

- El nombre de la función empieza con `test_`: es la convención que `pytest` usa para *descubrir* automáticamente qué funciones son pruebas (no necesita que las registren en ningún lado).
- `tmp_path` — Es un parámetro, pero no lo llena quien llama a la función: lo inyecta automáticamente `pytest`. Es una *fixture* nativa: en cada prueba, `pytest` crea una carpeta temporal única en el disco, y la entrega como un objeto `Path` ya listo para usar.
- **Arrange:** `archivo = tmp_path / "foto.jpg"` construye la ruta (todavía no existe el archivo); `archivo.write_text("contenido")` lo crea con ese contenido de texto.
- **Act:** en esta prueba, el "act" es simplemente llamar `clasificar(archivo)` dentro del `assert` mismo — no siempre el Act es una línea separada.
- **Assert:** `assert clasificar(archivo) == "Imagenes"`. Si `clasificar` devuelve cualquier otra cosa, la expresión es `False` y `pytest` marca la prueba como fallida, mostrando ambos valores para comparar.

2.2 Segunda prueba

```python
def test_organizar_mueve_archivo_a_subcarpeta(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")

    organizar_carpeta(tmp_path)

    assert (tmp_path / "Imagenes" / "foto.jpg").exists()
    assert not archivo.exists()
```

- **Arrange:** igual que antes, crea un archivo `foto.jpg` dentro de la carpeta temporal.
- **Act:** `organizar_carpeta(tmp_path)` — se llama sin el segundo argumento, así que `simulacion` toma su valor por defecto, `False`: esto ejecuta el movimiento real, pero dentro de la carpeta temporal, nunca en el disco real del estudiante.
- **Assert (dos verificaciones):**
  - `(tmp_path / "Imagenes" / "foto.jpg").exists()` — Construye la ruta esperada del archivo *después* de moverse, y verifica que exista ahí.
  - `not archivo.exists()` — Verifica que la variable `archivo`, que apunta a la ruta *original* (`tmp_path / "foto.jpg"`, sin la subcarpeta), ya **no** exista — es decir, que el archivo realmente se movió y no se copió.

2.3 Tercera prueba

```python
def test_organizar_en_modo_simulacion_no_mueve_nada(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")

    organizar_carpeta(tmp_path, simulacion=True)

    assert archivo.exists()
    assert not (tmp_path / "Imagenes").exists()
```

- Misma arrange que las anteriores.
- **Act:** esta vez se pasa `simulacion=True` explícitamente.
- **Assert (la prueba "espejo" de la anterior):**
  - `archivo.exists()` — el archivo original debe **seguir existiendo** (no se movió).
  - `not (tmp_path / "Imagenes").exists()` — la subcarpeta `Imagenes` ni siquiera debería haberse creado, porque en modo simulación el código nunca llega a la

línea `destino_carpeta.mkdir(...)`.

Estas tres pruebas, juntas, cubren: la lógica de clasificación sola, el comportamiento real completo, y que el modo seguro (`--dry-run`) realmente no toca nada — que es precisamente la garantía que se pedía en la diapositiva 5 del guion ("probar antes de confiar").

---

3. Los comandos de terminal, explicados

Comando	Qué hace
`.venv\Scripts\pip install pytest`	Instala pytest **dentro del entorno virtual** (no en el Python global de Windows)
`.venv\Scripts\pip freeze > requirements.txt`	Vuelca la lista de paquetes instalados y sus versiones exactas al archivo `requirements.txt`, sobrescribiéndolo
`.venv\Scripts\python organizador.py carpeta_prueba --dry-run`	Ejecuta el script, pasando `carpeta_prueba` como argumento posicional y activando la bandera de simulación
`.venv\Scripts\pytest -v`	Descubre y ejecuta todas las funciones `test_*` del proyecto; `-v` (verbose) muestra cada prueba por nombre con su resultado, en vez de solo un resumen
`.venv\Scripts\docformatter --in-place organizador.py`	Reformatea los docstrings de `organizador.py` para cumplir PEP 257; `--in-place` significa que sobrescribe el archivo (sin esa bandera, solo imprime el resultado en pantalla, sin guardar nada)
`git add organizador.py test_organizador.py requirements.txt`	Marca esos tres archivos para que entren en el próximo commit (los "agrega al área de preparación")
`git commit -m "feat: ..."`	Crea la "fotografía" del estado actual de los archivos marcados, con ese mensaje descriptivo
`git push origin main`	Sube los commits locales de la rama `main` al repositorio remoto en GitHub

---

Documento de apoyo docente — no forma parte de los entregables del estudiante. Uso previsto: preparación de clase antes de dictar el Laboratorio N.º 3.